ChatGPT

Optimization and Systems for Large-Scale Pre-Training

Introduction: Modern deep learning models, especially large language models, demand advanced optimization algorithms and distributed system techniques to train efficiently at scale. Conventional training setups struggle as model sizes reach billions of parameters, requiring innovations in optimizers, learning rate scheduling, precision handling, and parallelism. In this guide, we explore key optimizers (Adam, AdamW, Adafactor, etc.), learning-rate (LR) schedules, mixed-precision training, and distribution strategies like ZeRO and pipeline parallelism that enable multi-GPU pre-training of massive models. We blend foundational concepts with implementation notes and real-world case studies from recent research (e.g. Kingma & Ba 2014; Loshchilov & Hutter 2019; Micikevicius et al. 2018; Rajbhandari et al. 2020) to illustrate how **optimization and systems innovations** come together for large-scale model pre-training.

Adaptive Optimizers: Adam, AdamW, Adafactor, and Beyond

- **Adam (Adaptive Moment Estimation):** Introduced by Kingma & Ba (2014), Adam quickly became a go-to optimizer for deep learning. It computes adaptive learning rates for each parameter by combining ideas from momentum (first moment of gradients) and RMSProp (second moment of gradients). Adam is straightforward to implement and memory-efficient (requiring only first and second moment buffers), and it is **invariant to gradient rescaling**, making it robust for problems with noisy or sparse gradients [1]. Its default hyperparameters (e.g. β_1=0.9, β_2=0.999, ε=1e-8) work well across many tasks with little tuning [1]. Adam's ability to **handle non-stationary objectives and large-scale data** made it especially suitable for training very deep networks and transformers, hence its widespread use in pre-training of models like BERT and GPT.

- **AdamW (Adam with Decoupled Weight Decay):** Loshchilov & Hutter (2019) identified that the original Adam formulation coupled weight decay with the gradient update (essentially as L2 regularization in the loss). This coupling can interfere with Adam's adaptive learning rates and hinder convergence [2] [3]. **AdamW decouples weight decay from the gradient-based update**, applying a direct weight decay step on the parameters each update rather than as part of the loss gradient [3]. This simple modification ensures that weight decay (i.e. L2 regularization) behaves as intended – **shrinking weights independently of the adaptive gradients** – leading to more consistent regularization and often better generalization performance [4] [3]. Empirical evidence showed AdamW substantially improves Adam's performance on vision tasks, allowing it to match SGD with momentum on image classification where vanilla Adam fell short [5] [6]. Today, AdamW (or equivalent decoupled decay optimizers) is the default in many frameworks when weight decay is used.

- **Adafactor:** As models grow, even Adam/AdamW can become memory-intensive – they maintain two extra tensors (moment estimates) for every parameter. *Adafactor* (Shazeer & Stern 2018) is an Adam-derived optimizer designed for **memory efficiency at ultra-large scales**. Instead of storing a full second-moment tensor for each weight, Adafactor uses a factored representation: it maintains

rank-1 statistics (per-row and per-column sums of squared gradients) to approximate the second moment [7]. This reduces memory cost from $O(n \cdot m)$ to $O(n+m)$ for an $n \times m$ weight matrix [8] [9]. Adafactor forgoes the β_2 *momentum* buffer and even defaults to no bias correction, yet remains stable for very large batch training [7]. Google's **T5** (11B parameter Transformer) famously used Adafactor to enable feasible training on TPU pods [10]. In practice, Adafactor **maintains Adam-like performance while using far less memory**, making it ideal for enormous models where optimizer state memory is a bottleneck [11]. Its adaptive learning rate mechanism also relieves some hyperparameter tuning burden [12]. By removing "momentum" buffers and factorizing second-moment estimates, Adafactor helps scale training to models with tens of billions of parameters on memory-limited hardware.

- **Other Innovative Optimizers:** *Large-batch optimizers* and communication-efficient optimizers have also emerged to push scaling limits. **LAMB** (You et al. 2019) introduced layer-wise adaptive moments to enable **extremely large minibatch training**. Using LAMB, BERT-Large could be trained with batch sizes over 32k without loss of accuracy, cutting training time from 3 days to about **76 minutes on a TPU pod** [13]. LAMB achieves this by adapting the learning rate for each layer (avoiding the convergence drop that plain Adam suffers at huge batch sizes) and was key in the 76-minute BERT training demo [14] [15]. On the distributed front, the DeepSpeed library introduced **1-bit Adam and 1-bit LAMB** which **quantize and compress gradients** to drastically reduce communication in multi-node training [16]. These maintain similar convergence to full-precision Adam/LAMB but cut communication volume up to 26×, yielding over 6× throughput improvements for BERT-Large pre-training on bandwidth-constrained clusters [16]. Such innovations extend the reach of adaptive optimizers to regimes of *massive batch sizes* and *distributed training*, further speeding up large-scale pre-training.

Learning Rate Schedules and Warmup Strategies

Choosing how the learning rate changes during training is as important as the optimizer choice, especially for training large models. A poor learning-rate schedule can lead to divergence or slow convergence, whereas a good schedule can stabilize training and reach good solutions faster.

- **Warmup:** Large models and large batches often require a **warmup phase** where the learning rate starts very low and gradually increases over some initial portion of training. This helps avoid unstable jumps early on when model weights are random and gradients are large. For instance, the original Transformer paper used a linear warmup for the first 4,000 steps [17] (cited in many frameworks as default for transformers). Warmup mitigates the risk of blowing up the gradients at the start. Empirical studies found that warmup is especially crucial for Transformer architectures due to their initialization and layer normalization dynamics [18]. In practice, even a short warmup (e.g. a few hundred to a few thousand iterations) can resolve initial instability, allowing higher peak learning rates later [19] [20]. For example, Google's tuning guide notes that **just 1000 steps of warmup enabled stable training at a 10× higher learning rate** that would otherwise cause divergence [19]. Thus, most large-scale pre-training runs (BERT, GPT, etc.) use some LR warmup schedule (linear or cosine warmup) at the beginning.

- **LR Decay Schedules:** After warmup, the learning rate is usually decayed according to a schedule to ensure convergence. Common schedules include **step decay** (drop LR by some factor at fixed intervals), **linear decay** (gradually decrease LR to zero by the end of training), and **cosine annealing**

(smoothly decrease LR following a cosine curve, sometimes with restarts) [21]. In pre-training large models, a **linear decay after warmup** is very popular – for instance, BERT used a linear ramp up then linear decay to zero over 1e6 updates. **Cosine schedules** (Loshchilov & Hutter 2017) were introduced to allow aggressive initial training and periodic resets; they anneal the LR to a minimum and optionally jump back to a higher LR, which can help escape sharp local minima. Another effective strategy is the **One-Cycle Policy** (Smith 2018) where LR rises from a base value to a higher value mid-training and then decreases to a very low value by the end. This *1cycle* schedule often leads to faster convergence and better generalization in fewer epochs [22]. Notably, DeepSpeed supports one-cycle schedules and even a **Learning Rate Range Test** utility to help find a good LR range [23] [24] – these tools automate the process of tuning LR schedules for new tasks.

- **Large-Batch Considerations:** When scaling to very large batch sizes (thousands of examples per step), learning-rate schedules need special care. A known heuristic is to **scale the learning rate with the square root of batch size** and use a warmup [25]. Goyal et al. (2017) demonstrated this by scaling ResNet-50 training to batch 8K (LR scaled 10× with warmup), cutting training time from 29h to 1h. However, beyond a point, simply increasing LR with batch size fails – models might converge to worse minima. This motivated algorithms like LAMB (discussed above) and also **tuning of LR schedules** specifically for large batches. For example, LAMB's success in BERT training also required adjusting the decay schedule to accommodate the huge batches [14] [15]. Generally, with large batches one might use a **longer warmup and a more gradual decay**, ensuring that each phase of training (with its reduced noise in gradients) is given the appropriate learning rate. The takeaway is that **learning rate scheduling is crucial for stability and performance**: techniques like warmup combat early instability, and annealing schedules (linear or cosine) help fine-tune the model towards the end.

Mixed-Precision Training (FP16/BF16) and Efficiency

Training on lower-precision number formats is a major systems breakthrough that enables faster and more memory-efficient training without sacrificing model accuracy. **Mixed-precision training**, popularized by Micikevicius et al. (2018), uses a combination of 16-bit and 32-bit floating point representations to get the best of both worlds [26].

- **Benefits of FP16 Precision:** Using 16-bit floats (FP16) for most computations **cuts memory usage in half** (16 bits vs 32 bits) and allows **faster computation on modern hardware**. NVIDIA GPUs starting from Volta offer specialized Tensor Cores that can multiply matrices in FP16 with **up to 8× higher throughput** than FP32, while accumulating results in FP32 [27]. This yields significant speedups for matrix-heavy operations and reduces memory bandwidth pressure by halving data transfer per value [27]. In practice, mixed-precision training often speeds up training by ~2-4× and enables larger batch sizes or models to fit in GPU memory [27]. These benefits made FP16 training the standard for large-scale models – for instance, **almost all state-of-the-art language models (GPT-3, etc.) are trained using FP16 or bfloat16 mixed precision** to leverage these efficiency gains.

- **Techniques for Accuracy Preservation:** The challenge with 16-bit floats is their narrower dynamic range (FP16 can represent values ~[2^-24, 2^15], versus ~[2^-149, 2^127] for FP32). This can lead to **overflow or underflow** of gradients and tiny weight updates being lost. Micikevicius et al. proposed three key techniques to counter these issues so that FP16 training can match FP32 accuracy [26]:

- **Accumulating in FP32:** Any reduction operations like summing partial products are done in 32-bit. NVIDIA's tensor cores inherently multiply in half precision but accumulate in single precision, avoiding precision loss in summation [26] [28].
- **Loss Scaling:** The idea is to multiply the loss (and hence all gradients) by a scale factor (e.g. 128 or 1024) during backpropagation to **shift small gradient values into the representable range of FP16** [29] [30]. After computing gradients in FP16, we divide them by the same factor before the optimizer step [31]. This prevents gradients from vanishing to zero under FP16. Dynamic loss scaling techniques can adjust the factor on the fly if overflow is detected. Loss scaling proved essential – without it, FP16 training often diverges, whereas **with loss scaling, FP16 training can closely track FP32 accuracy** [32].
- **FP32 Master Weights:** We keep a copy of model weights in full 32-bit precision which is updated by the optimizer [33] [34]. Each iteration, a FP16 copy of these weights is used for forward and backward pass, but weight updates (which might be very small relative to the weight value) are applied to the high-precision master weights. This avoids the scenario where a tiny update is too small to change an FP16 weight (due to limited mantissa or large exponent disparity) [35] [34]. The master weights ensure no update information is lost, and we regenerate the FP16 weights from them every step.

With these strategies, Micikevicius et al. reported **no accuracy loss in a variety of models** – from image classification to language modeling – when comparing FP16 mixed-precision training to FP32 results [32] [26]. In fact, their big LSTM language model and object detection models all reached FP32-level perplexity or mAP using mixed precision (the only difference being that without loss scaling the models diverged, highlighting its importance [32]).

- **BFloat16:** An alternative 16-bit format, **bfloat16 (BF16)**, has become prevalent in newer hardware (TPUs and NVIDIA Ampere GPUs). BF16 uses 8 exponent bits (same as FP32) with 7 mantissa bits, trading precision for range. The wider dynamic range of BF16 largely avoids the need for loss scaling. Many recent large models (e.g. GPT-3, which was trained on TPU v3, and others on TPU v4) used bfloat16 for ease of implementation. On GPUs like A100/H100, one can train in BF16 mode to similar effect. Regardless of FP16 or BF16, the **key point is that half-precision training has unlocked massive speed and memory gains**, making it a cornerstone of modern large-scale training systems. Frameworks like PyTorch provide **Automatic Mixed Precision (AMP)** to automate these details (casting, scaling, master weights), so researchers can get the benefits with minimal code changes.

Distributed Training: Data Parallelism, Model/Pipeline Parallelism, and ZeRO

To train today's enormous models (with tens or hundreds of billions of parameters), one must go beyond a single GPU. **Distributed training** strategies have evolved to split the workload across multiple GPUs and even multiple nodes in a cluster. Each strategy addresses different bottlenecks (memory or compute) and often they are combined for best results.

- **Data Parallelism (DP):** The simplest form of multi-GPU training. Each GPU (or worker) holds a full copy of the model, and processes a different portion of the input data. Gradients are averaged (all-reduced) after each backward pass to keep models in sync. Data parallelism is easy to implement and scales compute almost linearly, but it does **not help with memory limits** – every GPU still needs to load the entire model. Pure data parallelism on current GPUs hits a wall when model size ≈ GPU

memory. For example, 32GB GPUs run out of memory around 1.4B parameter models under standard data parallel training [36] [37]. Thus, for models beyond this size, we need model partitioning techniques.

- **Model Parallelism:** In model (layer) parallelism, we *split the model's parameters across GPUs* so that each device holds only a slice of the full network. A classic approach is to split consecutive layers among different GPUs (pipeline parallelism, discussed next) or to split the *operations within a layer* (tensor-slicing model parallelism). NVIDIA's **Megatron-LM**, for instance, splits each Transformer layer's matrices across multiple GPUs (so each GPU computes a part of the dot-products) – this is **tensor parallelism**, a form of intra-layer model parallelism [38] [39]. Model parallelism reduces per-GPU memory usage (each GPU stores only part of the model), enabling larger models to fit. However, it introduces significant communication overhead: GPUs must exchange activations or gradients at layer boundaries or during matrix operations. This can become a scalability bottleneck, especially across nodes. The **Megatron-LM 8.3B model** (2019) used model parallelism within a node effectively, but attempts to scale it to multi-node (e.g. a 40B model across 2 nodes) showed **severely degraded efficiency (~5% of peak)** due to heavy inter-node communication [40]. Thus, while model parallelism is crucial, it often needs to be combined with other strategies to maintain high efficiency.

- **Pipeline Parallelism:** Pipeline parallelism (also called stage-based model parallelism) addresses the utilization issue of naive model parallelism. In pipeline parallelism, different GPUs are assigned different sets of consecutive layers (stages) of the model. A large mini-batch is then broken into smaller **micro-batches**, and these micro-batches are fed through the pipeline in a staggered manner [41] [42]. This way, all pipeline stages can work in parallel on different micro-batches, instead of one GPU waiting idle for another to finish its layer. **GPipe** (Google, 2018) demonstrated the power of pipeline parallelism by training giant networks that were previously infeasible [43] [44]. For example, GPipe allowed an AmoebaNet model with **557M parameters** to be split across 4 TPUv2 cores, achieving over 84% ImageNet accuracy [45]. Moreover, by partitioning an AmoebaNet across 8 accelerators (with recomputation to save memory), GPipe fit **1.8 billion parameters on 8 TPUs**, **25× more** than what a single device could handle [46]. It also nearly linearly increased throughput: using 4× more accelerators gave ~3.5× speedup vs. a 2-way split [47]. In another experiment, GPipe scaled a Transformer language model to **8 billion parameters on 16 TPUv3s**, using long sequences (1024 tokens) – and saw an **11× training speedup** compared to no-pipeline baseline [48]. These results were groundbreaking, proving that carefully designed pipeline parallelism can both **expand model size limits and accelerate training**. The downside of pipelining is the *"pipeline bubble"* – when the pipeline isn't full (at the start and end of each epoch, or if micro-batch count is too low), some devices idle. Recent pipelines use techniques like **pipeline parallelism with virtual micro-batches** and schedule tuning (e.g. PipeDream, Microsoft) to minimize bubble time and even allow asynchronous pipeline execution. In practice, one often combines pipeline parallelism with data parallelism: e.g. if you have 8 GPUs per node holding different pipeline stages, and replicate that setup across 4 nodes (data parallel), you utilize 32 GPUs effectively.

- **ZeRO: Sharded Data Parallelism:** To push memory limits further, Microsoft's **ZeRO (Zero Redundancy Optimizer)**, introduced by Rajbhandari et al. (2020), took data parallelism and made it *memory-efficient*. In standard DP, each of N GPUs holds a full copy of the model's parameters, gradients, and optimizer states (momentum, etc.) – that's a lot of redundancy. ZeRO **partitions these states across the GPUs instead of replicating them** [49]. In ZeRO Stage 1, optimizer states (e.g. Adam's m and v) are partitioned so each GPU only keeps a slice (1/N of each tensor) – yielding

up to **4× memory reduction with no increase in communication** [50]. Stage 2 adds gradient partitioning (each GPU only retains 1/N of the gradients, summing via reduce-scatter instead of allreduce), for **8× memory reduction vs. data parallel** (again, no extra communication vs. normal allreduce) [50]. Stage 3 further partitions the model parameters themselves: at any given time, each GPU holds only the pieces of the weights needed for the current computation, assembled just-in-time from shards (with some communication overhead) [51] [52]. With all three stages, **memory reduction scales with N** – e.g. with 64 GPUs, each has ~1/64 of the model, enabling **64× larger models** (with a modest increase in communication) [51]. The ZeRO approach **makes the total memory of the cluster available for the model** (plus optimizer), instead of being limited by a single GPU's memory. The authors showed that **with ZeRO, a model with *over 1 trillion* parameters could be trained on 1024 GPUs** (since 1 trillion * 16 bytes per param (FP16 weight + FP16 grad + FP32 optimizer states) ≈ 16 TB total, which divided by 1024 GPUs is 16 GB per GPU – fitting comfortably in memory) [52]. While 1 trillion was a theoretical extrapolation, they *implemented* ZeRO to train **models over 100 billion parameters on 400 GPUs**, achieving *super-linear speedups* (15 PetaFLOP/s throughput) thanks to better parallel efficiency [53]. In fact, ZeRO enabled training a 13B model **without any model parallelism** (previously impossible on that hardware) and was used to train **Microsoft's Turing-NLG 17B model**, the world's largest language model at its release [54]. This was a massive leap, demonstrating that *memory, not compute, was the chief limiter*, and by eliminating memory redundancy, one can scale to new heights. DeepSpeed's implementation of ZeRO (now in stages 1-3) seamlessly integrates with PyTorch, making it relatively easy to leverage these gains.

- **ZeRO-Offload and CPU Memory:** A further extension, **ZeRO-Offload**, goes one step beyond by partitioning states **and** moving much of the memory load to CPU RAM when GPU memory is scarce. DeepSpeed's ZeRO-Offload can keep optimizer states on the CPU and leverage CPU memory swap-in/out, enabling e.g. **13 billion parameter models to train on a single GPU** (using CPU RAM as overflow) [55]. This is about 10× larger than what vanilla approaches allowed on one GPU, and it still sustains high throughput (~30 TFLOPs on a V100) by overlapping computation and data transfer efficiently [55]. This technique **democratizes large-model training**, allowing researchers without big GPU clusters to experiment with multi-billion-parameter models [56].

- **3D Parallelism (Hybrid):** The ultimate scaling uses **combinations of data, model, and pipeline parallelism** – often referred to as *"3D parallelism."* DeepSpeed and Megatron-LM have been integrated to support this: e.g. Megatron-Deepspeed applies tensor model parallel *within* each pipeline stage, uses pipeline parallel across stages, and then applies ZeRO-powered data parallel across multiple replicas of the pipeline [38] [39]. This yields a highly scalable mix that can train models with **trillions of parameters** given sufficient hardware [57]. For example, DeepSpeed reported scaling **to over a trillion parameters using 3D parallelism** in a prototype [57]. The idea is to **use model parallel (tensor slicing) to split each layer across a few GPUs**, use **pipeline parallel to stack more layers than one GPU can hold**, and use **ZeRO data parallel to replicate this whole pipeline across nodes efficiently**. By reducing the degree of model parallel needed (which can be inefficient at scale) and instead using more ZeRO sharding + pipeline, DeepSpeed achieved up to **10× faster training for models in the 1.5B–100B range** compared to pure model parallelism [58]. An example case: **Megatron-DeepSpeed** was used by the BigScience project to train the 176B-parameter BLOOM model on 384 GPUs, combining all these parallelism techniques. Although such hybrid setups are complex, libraries abstract a lot of it: DeepSpeed's API allows a user to enable these features with minimal code changes (just a config file selection for ZeRO stage, pipeline degree, etc.) [59] [60].

Case Study – BERT-Large and GPT-2: To appreciate these system optimizations, consider BERT-Large (340M params) pre-training. Originally, BERT-Large took over 3 days on 16 TPUs. Using optimizers like LAMB with huge batches, and massive parallelism (1024 GPUs with mixed precision), Microsoft demonstrated BERT-Large training in **44 minutes** to the same accuracy [61]. This was done with DeepSpeed, leveraging **gradient accumulation and ZeRO to use a batch size of 32k**, and 1-bit Adam to reduce communication, achieving unprecedented speed [62] [61]. Similarly, GPT-2 (1.5B params) was trained 3.75× faster on Azure GPUs with DeepSpeed compared to NVIDIA's prior best (Megatron) by using better parallelism and optimizers [63]. These examples underscore how **advanced optimizers + efficient parallelism yield state-of-the-art training performance**.

Conclusion

Training huge models is no longer just about more GPUs – it requires smart optimization algorithms and system design. **Adaptive optimizers** like Adam and its variants (AdamW fixing weight decay, Adafactor saving memory) provide robust and efficient updates for deep networks, while specialized techniques like LAMB target the large-batch regime to shorten training times [13]. **Learning rate schedules** (warmup and decay policies) are critical for guiding training to convergence, especially in the context of very large models and batches where instability looms without warmup [19]. **Mixed-precision training** has become a standard tool to squeeze more performance out of hardware, leveraging FP16/BF16 to significantly speed up training while matching full-precision accuracy [27] [26]. On the systems side, **parallelism and memory optimization techniques** are what make training a 100B+ parameter model even possible: pipeline and model parallelism split the model across GPUs to overcome memory limits [46], and innovations like **ZeRO** sharding eliminate redundant memory usage, allowing linear scaling of model size with more devices [49] [52]. These methods, combined in frameworks like **DeepSpeed**, have already enabled record-breaking models (trillion-parameter scale) and dramatically reduced training times for smaller ones. The field continues to evolve (e.g. newer optimizer variants, better parallel scheduling, efficient fine-tuning methods for giant models), but the strategies outlined here form the *foundation* of modern large-scale pre-training. By blending algorithmic tweaks (optimizers & LR schedules) with system-level techniques (mixed precision & parallelism), we can train "unthinkably" large neural networks with reasonable time and resource costs – a synergy of optimization and systems that is driving the current revolution in AI.

Sources: The concepts and case studies discussed are drawn from seminal papers and recent advancements, including Kingma & Ba (2014) for Adam [1], Loshchilov & Hutter (2019) for AdamW [5], Shazeer & Stern (2018) for Adafactor [7], Micikevicius et al. (2018) for mixed precision training techniques [26], and Rajbhandari et al. (2020) for ZeRO and large-scale training results [53] [52], among other sources as cited throughout.

---

1 [1412.6980] Adam: A Method for Stochastic Optimization
https://arxiv.org/abs/1412.6980

2 3 4 Adam vs. AdamW: Understanding Weight Decay and Its Impact on Model Performance | by Ahmed Yassin | Medium
https://yassin01.medium.com/adam-vs-adamw-understanding-weight-decay-and-its-impact-on-model-performance-b7414f0af8a1

5 6 [1711.05101] Decoupled Weight Decay Regularization
https://arxiv.org/abs/1711.05101

7 10 11 12 Adafactor - Cornell University Computational Optimization Open Textbook - Optimization Wiki
https://optimization.cbe.cornell.edu/index.php?title=Adafactor

8 9 Adafactor: Adaptive Learning Rates with Sublinear Memory Cost
https://arxiv.org/pdf/1804.04235

13 14 15 25 [1904.00962] Large Batch Optimization for Deep Learning: Training BERT in 76 minutes
https://ar5iv.labs.arxiv.org/html/1904.00962

16 22 23 24 49 55 56 57 58 59 60 61 62 63 Training Overview and Features - DeepSpeed
https://www.deepspeed.ai/training/

17 18 [D] Learning rate warm up for transformers and more - Reddit
https://www.reddit.com/r/MachineLearning/comments/eumvgw/d_learning_rate_warm_up_for_transformers_and_more/

19 20 FAQ | Machine Learning | Google for Developers
https://developers.google.com/machine-learning/guides/deep-learning-tuning-playbook/faq

21 Why Warmup the Learning Rate? Underlying Mechanisms ... - arXiv
https://arxiv.org/html/2406.09405v1

26 27 28 29 30 31 32 33 34 35 Mixed-Precision Training of Deep Neural Networks | NVIDIA Technical Blog
https://developer.nvidia.com/blog/mixed-precision-training-deep-neural-networks/

36 37 40 50 51 52 53 54 [1910.02054] ZeRO: Memory Optimizations Toward Training Trillion Parameter Models
https://ar5iv.labs.arxiv.org/html/1910.02054

38 39 Pre-training a large language model with Megatron-DeepSpeed on multiple AMD GPUs — ROCm Blogs
https://rocm.blogs.amd.com/artificial-intelligence/megatron-deepspeed-pretrain/README.html

41 42 43 44 45 46 47 48 Introducing GPipe, an Open Source Library for Efficiently Training Large-scale N
https://research.google/blog/introducing-gpipe-an-open-source-library-for-efficiently-training-large-scale-neural-network-models/